HPC Workload Characterisation Survey

Purpose, process and a guide to the form

HPC Users' Committee

October 26, 2026

IISER Bhopal

Purpose and scope

Why this survey

- Understand the **computing workloads** of research groups across the institute.
- See how well **Gargi** and **Raman** serve present needs, and where they do not.
- Identify problems — resource limits, usage policies, software, storage — that **can be resolved**.
- Plan for **growth** over the next few years.

The principle

You describe what you observe. The committee applies agreed, transparent tests. You are not asked to justify your research or to judge your own priority.

Scope

The survey covers

- what you run, and how
- the resources your jobs use
- problems you have met, on any system
- which of those arose on Gargi or Raman
- expected growth

It does not

- rank groups or research areas
- assign queues or priorities
- decide who may use the clusters
- treat “Don’t know” as “No”

Experience on national facilities, collaborators’ clusters or the cloud counts equally. Little or no use of the present clusters is *not* evidence of little need.

Issues the survey is designed to capture

Some situations users have described, in general terms:

- A long run stops at the wall-time limit and restarts from an earlier point.
- Short test jobs wait in the queue as long as large production runs.
- A calculation needs more nodes than any queue allows, so it runs elsewhere.
- A required library or version is not available on the cluster.
- Intermediate files outgrow the scratch space available to a job.
- Work moves off a cluster after a change in its usage rules.

The form records such issues as facts, **separately for Gargi and for Raman**, so that each can be traced to a resource, a policy or a missing piece of software.

Who fills it in

Who fills in the form, and who can help

One submission per research group

- the PI, or
- a group member who runs the calculations — often the person who knows the details best

The PI receives a copy of the confirmation.

Help is available

- A volunteer in each department can help with the technical questions.
- Department contacts will be informed separately

“Don’t know” is always a valid answer

Estimates are welcome when marked as estimates. Incomplete technical knowledge does not disadvantage a submission.

Before you start

Useful to have at hand

- the main codes or applications you run
- for the heaviest job of each: nodes, cores or GPUs, memory, wall time
- roughly how many such jobs per year
- storage you use, and any problems you have met

Practicalities

- about **20–30 minutes**
- fill the sections **in order**: answers decide later questions
- the draft is saved in your browser as you go

Terms used in the form

Terms (1/2)

Job	One submission to the scheduler (one `sbatch`).
Node	One server in the cluster; it has many cores, and may have GPUs.
Core	One processing unit. A job's cores = nodes × cores used per node.
Wall time	How long a job runs, from start to finish — *excluding* time waiting in the queue.
Queue wait	Time from submission until the job starts running.
Calendar time	From submitting the first job to the last one finishing, waits included.

Example: a job submitted at 9:00 starts at 13:00 and ends at 21:00.

Queue wait = 4 h Wall time = 8 h Calendar time = 12 h

Terms (2/2)

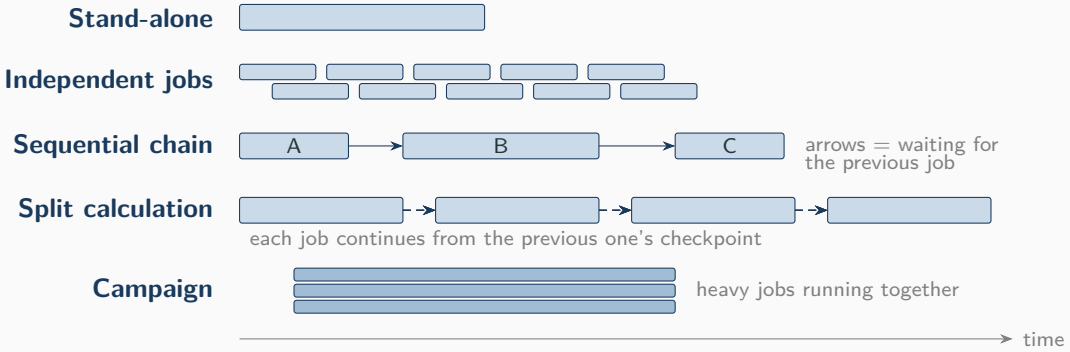
Core-hours	Cores $\times$ wall time. 2 nodes $\times$ 128 cores for 10 h $=$ 2,560 core-hours.
GPU-hours	GPUs $\times$ wall time. 4 GPUs for 6 h $=$ 24 GPU-hours.
Wall-time limit	The longest a job may run in a queue. Present limits: 72 h (multi-node CPU), 168 h (single node), 24 h (GPU).
Checkpoint/restart	A job saves its state and can continue later without repeating completed work.
Home / scratch	Home keeps files between jobs; scratch is fast working space for files a job reads and writes while it runs.
Doubling test	Run the same calculation on N and $2N$ cores and compare wall times (e.g. 10 h $\rightarrow$ 6 h is 0.6$\times$).

How jobs are organised

Pattern	Meaning	For example
Stand-alone	single calculations, each complete in one job	one simulation or analysis per job
Independent jobs	many jobs that do not depend on each other	parameter sweeps, ensembles, screening
Sequential chain	each job needs the previous job's output	preparation $\rightarrow$ main run $\rightarrow$ analysis, as separate jobs
Split calculation	one calculation longer than the limit, continued over several jobs	a long run restarted from its checkpoint
Campaign	many heavy jobs running together toward one result	coupled runs that must progress at the same time

Several can apply. Steps run inside *one* job script are one stand-alone job.

The patterns at a glance



The form

Layout of the form

	Section	Purpose
A	About your group	contact details; who is filling in the form
B	Your HPC experience	systems used; problems met — anywhere, then on Gargi and on Raman
C	Your workloads	one entry per code and kind of job: size, wall time, jobs per year, pattern
D	Details for each code	short follow-up panels, only where an entry calls for them
E	Storage, testing, interactive use	home space; short test and interactive sessions
F	Queue and turnaround	only if queue waits or capacity problems were reported
G	Growth and closing	expected growth; optional help with benchmarks; comments

Required answers are marked *; each has “Don’t know” where it applies.

Section B — problems on each cluster

Which of these did your group face on Gargi, including when deciding whether to run there? *

Only the problems you ticked above are listed. Gargi and Raman are asked separately because their hardware and usage policies differ.

- [x] Jobs stopped by the wall-time limit before completion
- [x] Jobs needed more nodes or cores than allowed
- [ ] Software or libraries we need are not available on Gargi
- [ ] None of these on Gargi
- [ ] We have not used Gargi or considered running there
- [ ] Don't know / not aware

- First: problems met on *any* system.
- Then: which of them arose on **Gargi** — and, in a separate question, on **Raman**.
- Their hardware and usage policies differ.

Section B — did the problems keep work away?

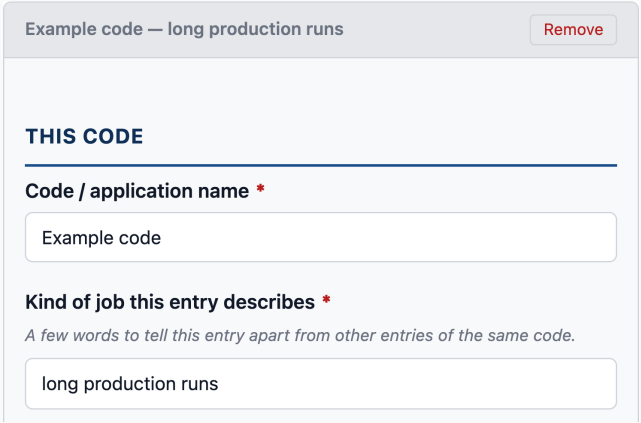
Did these problems stop your group from running some of its work on Gargi? *

For example, jobs that no queue allows, or software that is missing. Work that went to another system, or was not run at all, counts.

- (●) Yes — some of our work was run elsewhere or not at all
- ( ) Yes — we do not use Gargi because of them
- ( ) No
- ( ) Don't know / not aware

- Asked after each cluster's problems.
- Work run elsewhere, or not run at all, is recorded — it is demand the clusters did not see.

Section C — one entry per code and kind of job



- The same code can have several entries for quite different kinds of jobs.
- Describe the **heaviest** job of each entry; the committee works out the totals.
- Jobs run elsewhere count too.

Section C — how the jobs are organised

Which patterns describe how these jobs are organised? Select all that apply. *

Tick all that describe these jobs, e.g. a chain whose longest step needs restarts. Count it as a chain only if the steps are submitted as separate jobs; steps run inside one job script make one stand-alone job. If the same code runs kinds of job with different set-ups (say, many small screening jobs and a few large production runs), give each its own entry.

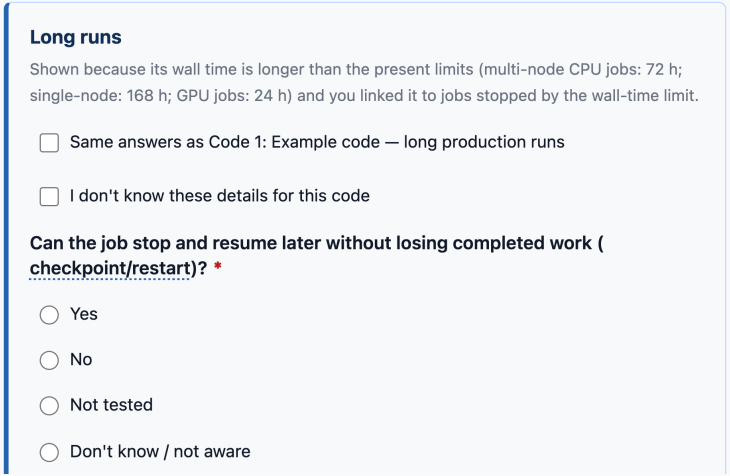
- [ ] Single stand-alone calculations
- [ ] Many independent jobs (parameter sweeps, ensembles, screening)
- [ ] Sequential chain — each job needs the previous job's output (e.g. preparation → main calculation → analysis, submitted as separate jobs)
- [x] One calculation that runs longer than the wall-time limit allows, so it is split into several jobs (restarts)
- [ ] Many heavy jobs running together toward one result (coordinated campaign)
- [ ] Don't know / not aware

Section D — follow-up panels

Panel	Opens when an entry ...	Asks about
Job organisation	uses independent jobs, a chain, a split calculation or a campaign	batch size; length of a sequence; campaign size
Long runs	exceeds a wall-time limit, or is linked to jobs stopped by it	checkpoint/restart; jobs stopped
More cores	is long or large, or is linked to node-count limits	effect of doubling cores; measured runs
Memory	needs more than a standard node, or ran out of memory	standard nodes; spreading memory
GPU	runs on GPUs, or is linked to GPU access	GPU model, memory, interconnect
Files and scratch	reads or writes large volumes, or is linked to scratch	I/O pattern; how long files stay

Each panel appears once per entry, whatever opened it, and says why.

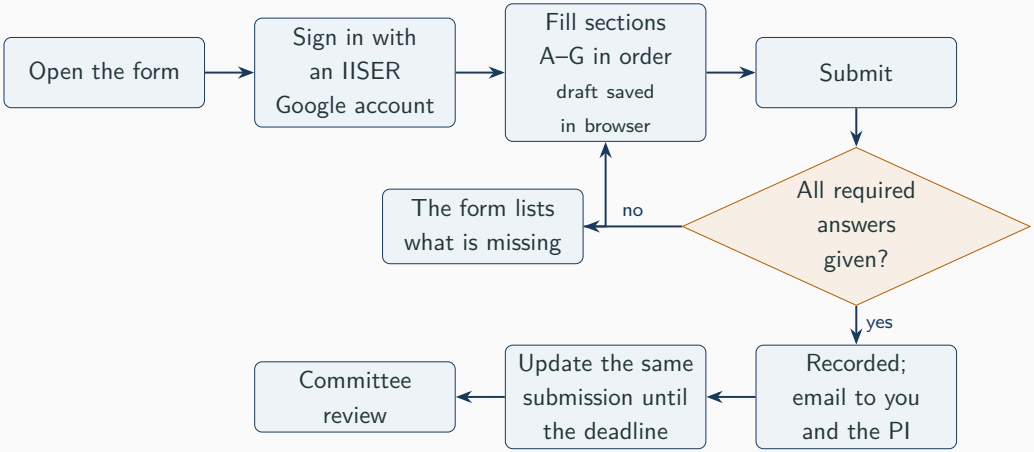
Section D — a panel



- "Shown because …" gives every reason.
- *Same answers as …* reuses an earlier entry of the same code.
- *I don't know these details* is a valid answer.

Submission and evaluation

Submitting



Updating uses the same browser and device on which the form was submitted.

How the committee uses the responses



Triage suggestions

1 — characterised	the information is sufficient; nothing further needed
2 — clarification	a short follow-up resolves a gap or an inconsistency
3 — technical look	an unusual requirement is discussed with the group, with proportionate evidence

Principles of the evaluation

- No scores, no rankings, no automatic queue assignment.
- "Don't know" and missing data are never counted against a group.
- Answers that do not fit together become **questions to the group**, not findings.
- Comparisons with other sources (e.g. scheduler records, code documentation) are made by people, openly.
- Unusual requirements call for evidence in proportion — just as ordinary ones are characterised by evidence.

Timeline and contacts

- The form will be shared in a few days
- Department contacts will be informed when it opens

Thank you for helping us understand the range of computing work at IISER Bhopal, so that services can be designed to serve it fairly.